

REFORMER: THE EFFICIENT TRANSFORMER

Nikita Kitaev*

U.C. Berkeley & Google Research
kitaev@cs.berkeley.edu

Łukasz Kaiser*

Google Research
{lukaszkaizer, levskaya}@google.com

Anselm Levskaya

Google Research

ABSTRACT

Large Transformer models routinely achieve state-of-the-art results on a number of tasks but training these models can be prohibitively costly, especially on long sequences. We introduce two techniques to improve the efficiency of Transformers. For one, we replace dot-product attention by one that uses locality-sensitive hashing, changing its complexity from $O(L^2)$ to $O(L \log L)$, where L is the length of the sequence. Furthermore, we use reversible residual layers instead of the standard residuals, which allows storing activations only once in the training process instead of N times, where N is the number of layers. The resulting model, the Reformer, performs on par with Transformer models while being much more memory-efficient and much faster on long sequences.

1 INTRODUCTION

The Transformer architecture (Vaswani et al., 2017) is widely used in natural language processing and yields state-of-the-art results on a number of tasks. To obtain these results, researchers have resorted to training ever larger Transformer models. The number of parameters exceeds 0.5B per layer in the largest configuration reported in (Shazeer et al., 2018) while the number of layers goes up to 64 in (Al-Rfou et al., 2018). Transformer models are also used on increasingly long sequences. Up to 11 thousand tokens of text in a single example were processed in (Liu et al., 2018) and when processing other modalities, like music (Huang et al., 2018) and images (Parmar et al., 2018), even longer sequences are commonplace. These large-scale long-sequence models yield great results but strain resources to the point where some argue that this trend is breaking NLP research¹. Many large Transformer models can only realistically be trained in large industrial research laboratories and such models trained with model parallelism cannot even be fine-tuned on a single GPU as their memory requirements demand a multi-accelerator hardware setup even for a single training step.

Do large Transformer models fundamentally require such huge resources or are they simply inefficient? Consider the following calculation: the 0.5B parameters used in the largest reported Transformer layer account for 2GB of memory. Activations for 64K tokens with embedding size 1024 and batch size 8 account for $64K \times 1K \times 8 = 0.5B$ floats, requiring another 2GB of memory. If our memory use was only per-layer, then we should fairly easily fit a large Transformer even on sequences of length 64K on a single accelerator. Further, the whole corpus used to train BERT only requires 17GB to store. Why is it then that we cannot even fine-tune these models on single machines?

The above estimate includes only per-layer memory and input activations cost and does not take into account the following major sources of memory use in the Transformer.

- Memory in a model with N layers is N -times larger than in a single-layer model due to the fact that activations need to be stored for back-propagation.
- Since the depth d_{ff} of intermediate feed-forward layers is often much larger than the depth d_{model} of attention activations, it accounts for a large fraction of memory use.
- Attention on sequences of length L is $O(L^2)$ in both computational and memory complexity, so even for a single sequence of 64K tokens can exhaust accelerator memory.

*Equal Contribution

¹<https://hackingsemantics.xyz/2019/leaderboards/>

We introduce the Reformer model which solves these problems using the following techniques:

- Reversible layers, first introduced in Gomez et al. (2017), enable storing only a single copy of activations in the whole model, so the N factor disappears.
- Splitting activations inside feed-forward layers and processing them in chunks removes the d_{ff} factor and saves memory inside feed-forward layers.
- Approximate attention computation based on locality-sensitive hashing replaces the $O(L^2)$ factor in attention layers with $O(L \log L)$ and so allows operating on long sequences.

We study these techniques and show that they have negligible impact on the training process compared to the standard Transformer. Splitting activations in fact only affects the implementation; it is numerically identical to the layers used in the Transformer. Applying reversible residuals instead of the standard ones does change the model but has a negligible effect on training in all configurations we experimented with. Finally, locality-sensitive hashing in attention is a more major change that can influence the training dynamics, depending on the number of concurrent hashes used. We study this parameter and find a value which is both efficient to use and yields results very close to full attention.

We experiment on a synthetic task, a text task (enwik8) with sequences of length 64K and an image generation task (imagenet-64 generation) with sequences of length 12K. In both cases we show that Reformer matches the results obtained with full Transformer but runs much faster, especially on the text task, and with orders of magnitude better memory efficiency.

2 LOCALITY-SENSITIVE HASHING ATTENTION

Dot-product attention. The standard attention used in the Transformer is the scaled dot-product attention (Vaswani et al., 2017). The input consists of queries and keys of dimension d_k , and values of dimension d_v . The dot products of the query with all keys are computed, scaled by $\sqrt{d_k}$, and a softmax function is applied to obtain the weights on the values. In practice, the attention function on a set of queries is computed simultaneously, packed together into a matrix Q . Assuming the keys and values are also packed together into matrices K and V , the matrix of outputs is defined as:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \quad (1)$$

Multi-head attention. In the Transformer, instead of performing a single attention function with d_{model} -dimensional keys, values and queries, one linearly projects the queries, keys and values h times with different, learned linear projections to d_k , d_k and d_v dimensions, respectively. Attention is applied to each of these projected versions of queries, keys and values in parallel, yielding d_v -dimensional output values. These are concatenated and once again projected, resulting in the final values. This mechanism is known as multi-head attention.

Memory-efficient attention. To calculate the memory use of the attention mechanism, let us focus on the attention computation from Equation 1. Let us assume that Q , K and V all have the shape $[batch_size, length, d_{model}]$. The main issue is the term QK^T , which has the shape $[batch_size, length, length]$. In the experimental section we train a model on sequences of length $64K$ – in this case, even at batch-size of 1, this is a $64K \times 64K$ matrix, which in 32-bit floats would take 16GB of memory. This is impractical and has hindered the use of the Transformer for long sequences. But it is important to note that the QK^T matrix does not need to be fully materialized in memory. The attention can indeed be computed for each query q_i separately, only calculating $\text{softmax}\left(\frac{q_i K^T}{\sqrt{d_k}}\right)V$ once in memory, and then re-computing it on the backward pass when needed for gradients. This way of computing attention may be less efficient but it only uses memory proportional to $length$. We use this memory-efficient implementation of attention to run the full-attention baselines presented in the experimental section.

Where do Q , K , V come from? The multi-head attention described above operates on keys, queries and values, but usually we are only given a single tensor of activations A of the shape $[batch_size, length, d_{model}]$ – e.g., coming from embedding the tokens in a sentence into vectors.

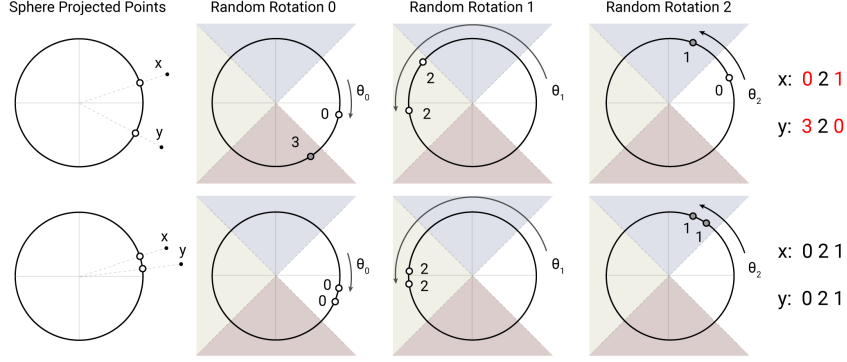


Figure 1: An angular locality sensitive hash uses random rotations of spherically projected points to establish buckets by an argmax over signed axes projections. In this highly simplified 2D depiction, two points x and y are unlikely to share the same hash buckets (above) for the three different angular hashes unless their spherical projections are close to one another (below).

To build Q, K and V from A, the Transformer uses 3 different linear layers projecting A into Q, K and V with different parameters. For models with LSH attention, we want queries and keys (Q and K) to be identical. This is easily achieved by using the same linear layer to go from A to Q and K, and a separate one for V. We call a model that behaves like this a shared-QK Transformer. It turns out that sharing QK does not affect the performance of Transformer, even if we additionally normalize the length of the keys K, as we show in the experimental Section 5.

Hashing attention. For the LSH attention, we start with two tensors, $Q=K$ and V of the shape $[batch_size, length, d_{model}]$. We keep the multi-head mechanism intact and focus on the attention computation from Equation 1. As already mentioned, the main issue is the term QK^T , which has the shape $[batch_size, length, length]$. But note that we are actually only interested in $\text{softmax}(QK^T)$. Since softmax is dominated by the largest elements, for each query q_i we only need to focus on the keys in K that are closest to q_i . For example, if K is of length 64K, for each q_i we could only consider a small subset of, say, the 32 or 64 closest keys. That is much more efficient, but how can we find the nearest neighbors among the keys?

Locality sensitive hashing. The problem of finding nearest neighbors quickly in high-dimensional spaces can be solved by locality-sensitive hashing (LSH). A hashing scheme that assigns each vector x to a hash $h(x)$ is called locality-sensitive if nearby vectors get the same hash with high probability and distant ones do not. In our case, we actually only require that nearby vectors get the same hash with high probability and that hash-buckets are of similar size with high probability.

We achieve this by employing random projections as follows (see Figure 1). To get b hashes, we first fix a random matrix R of size $[d_k, b/2]$. We then define $h(x) = \arg \max([xR; -xR])$ where $[u; v]$ denotes the concatenation of two vectors. This method is a known LSH scheme (Andoni et al., 2015) and is easy to implement and apply to batches of vectors.

LSH attention. Knowing our LSH scheme and the general idea of hashing attention, we will now formalize the LSH attention we use in this paper. We first rewrite the equation for normal attention, (1), for a single query position i at a time:

$$o_i = \sum_{j \in \mathcal{P}_i} \exp(q_i \cdot k_j - z(i, \mathcal{P}_i)) v_j \quad \text{where } \mathcal{P}_i = \{j : i \geq j\} \quad (2)$$

We introduce the notation $\mathcal{P}_i$ to represent the set that the query at position i attends to, and z to denote the partition function (i.e. the normalizing term in the softmax). For clarity, we also omit scaling by $\sqrt{d_k}$.

For batching purposes we typically perform attention over a larger set $\tilde{\mathcal{P}}_i = \{0, 1, \dots, l\} \supseteq \mathcal{P}_i$ while masking out elements not in $\mathcal{P}_i$:

$$o_i = \sum_{j \in \tilde{\mathcal{P}}_i} \exp(q_i \cdot k_j - m(j, \mathcal{P}_i) - z(i, \mathcal{P}_i)) v_j \quad \text{where } m(j, \mathcal{P}_i) = \begin{cases} \infty & \text{if } j \notin \mathcal{P}_i \\ 0 & \text{otherwise} \end{cases} \quad (3)$$